第 6 讲：链表——通过引用连接数据

本地交互讲义：打开 @@CODE0@@。网页为单个本地文件，不需要联网或登录。

0. 本讲的核心问题

数组通过下标访问元素，但在开头插入一个元素时，后面的元素通常都要移动。

能否让元素不依赖连续位置，而是让每个元素指出“下一个元素在哪里”？

这就是链表：

```
[  3 | next] → [  7 | next] → [  9 | None]
```

链表题的难点通常不是数学，而是：

> 修改引用之前，必须确保仍然能找到后续节点。

---

1. 节点与单链表

Python 中常见节点定义：

```
[python]
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

创建链表：

```
[python]
third = ListNode(9)
second = ListNode(7, third)
head = ListNode(3, second)
```

结构：

```
head
 ↓
 3 → 7 → 9 → None
```

`head` 不是整条链表本身，而是对第一个节点的引用。通过不断读取 `.next` 才能访问后续节点。

空链表表示为：

```
[python]
head = None
```

LeetCode 通常已经提供 `ListNode`，你只需要编写操作链表的方法。

---

2. 链表与数组的区别

操作	动态数组 `list`	单链表
按下标访问第 `i` 个	`O(1)`	`O(i)`
已知节点后插入	通常涉及移动	`O(1)`
已知前驱后删除	通常涉及移动	`O(1)`
查找某个值	`O(n)`	`O(n)`
内存组织	逻辑上连续槽位	节点通过引用连接

链表的 `O(1)` 插入有一个前提：已经拿到正确位置的节点。如果还要先遍历寻找位置，总时间仍可能是 `O(n)`。

---

3. 遍历链表

```
[python]
def linked_values(head):
    result = []
    current = head

    while current is not None:
        result.append(current.val)
        current = current.next

    return result
```

循环不变量：

> `current` 指向下一个尚未处理的节点，`result` 保存此前节点的值。

常见简写：

常见简写是 `while current:`。

它与 `while current is not None` 在这里等价。

不要写：

不要写成 `while current.next:`。

这会漏掉最后一个节点，并且空链表时访问 `None.next` 会报错。

---

4. 在已知节点后插入

原结构：

```
A → C
```

插入 `B`：

```
[python]
b = ListNode("B")
b.next = a.next
a.next = b
```

结果：

```
A → B → C
```

顺序很重要。先让 `B` 指向原来的 `C`，再让 `A` 指向 `B`。

如果先执行：

```
[python]
a.next = b
```

而没有提前保存或连接 `C`，就可能丢失原后继节点。

---

5. 删除下一个节点

原结构：

```
A → B → C
```

如果已经拿到 `A`，删除 `B`：

```
[python]
a.next = a.next.next
```

变成：

```
A ———→ C
```

Python 中如果没有其他引用指向 `B`，它最终会由垃圾回收处理。与 C 不同，这里不需要手动 `free`。

---

6. 虚拟头节点 dummy

删除或合并链表时，头节点经常是特殊情况。

例如删除所有等于 `target` 的节点：

```
1 → 1 → 2 → 3
```

如果开头的节点需要删除，`head` 本身会变化。虚拟头节点把头部操作转化为普通的“删除下一个节点”：

```
[python]
def remove_elements(head, target):
    dummy = ListNode(0, head)
    current = dummy

    while current.next is not None:
        if current.next.val == target:
            current.next = current.next.next
        else:
            current = current.next

    return dummy.next
```

为什么删除后不移动 `current`?

因为新的 `current.next` 也可能等于 `target`，需要继续检查。

循环不变量：

> `current` 之前的链表已经处理完毕且不包含目标值；`current.next` 是下一个待检查节点。

---

7. 核心题：反转链表

原链表：

```
1 → 2 → 3 → None
```

目标：

```
None ← 1 ← 2 ← 3
                ↑
              new head
```

7.1 为什么需要三个引用

处理 `current` 时，要把它的 `.next` 改成前一个节点。但修改之前必须保存原来的下一个节点，否则剩余链表会丢失。

```
[python]
def reverse_list(head):
    previous = None
    current = head

    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node

    return previous
```

四步必须理解：

```
1.      next_node = current.next
2.      current.next = previous
3. previous
4. current        next_node
```

7.2 循环不变量

每轮开始时：

- `previous` 是已经反转部分的头节点；
- `current` 是尚未处理部分的头节点；
- 两部分合起来包含原链表全部节点，没有丢失节点。

循环结束时 `current is None`，说明未处理部分为空，`previous` 就是完整反转链表的头。

7.3 复杂度

- 每个节点处理一次：`O(n)` 时间；
- 只使用固定数量引用：`O(1)` 额外空间。

---

8. 合并两个有序链表

```
list1: 1 → 3 → 5
list2: 2 → 4 → 6
```

每次选择两个当前节点中较小的一个接到结果尾部。

```
[python]
def merge_two_lists(list1, list2):
    dummy = ListNode()
    tail = dummy

    while list1 is not None and list2 is not None:
        if list1.val <= list2.val:
            tail.next = list1
            list1 = list1.next
        else:
```

```
                tail.next = list2
                list2 = list2.next

            tail = tail.next

        if list1 is not None:
            tail.next = list1
        else:
            tail.next = list2

        return dummy.next
```

不变量：

> `dummy.next` 到 `tail` 是已经确定的有序前缀；`list1` 和 `list2` 分别指向两个链表尚未合并的第一个节点。

尾部剩余链表已经有序，可以整体接上，不需要逐个复制节点。

时间 `O(m+n)`，额外空间 `O(1)`，因为复用了原节点。

---

9. 快慢指针：寻找中点

让：

- `slow` 每次走一步；
- `fast` 每次走两步。

当 `fast` 到达末尾时，`slow` 走了大约一半。

```
[python]
def middle_node(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    return slow
```

偶数个节点时，这个版本返回第二个中间节点：

```
1 → 2 → 3 → 4
        ↑
        3
```

循环条件顺序很重要：只有确认 `fast` 非空，才能读取 `fast.next`。

---

10. 快慢指针：判断环

有环链表：

```
1 → 2 → 3 → 4
        ↑       ↓
        └───────┘
```

如果快慢指针进入环：

- `slow` 每轮走 1 步；
- `fast` 每轮走 2 步；

- 快指针相对慢指针每轮接近 1 步。

在有限长度的环中，它们最终必然相遇。

```
[python]
def has_cycle(head):
    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

        if slow is fast:
            return True

    return False
```

这里使用：

```
[python]
slow is fast
```

因为要判断是否为同一个节点对象，而不是节点值是否相同。不同节点完全可以保存相同的 `val`。

- 时间：`O(n)`；
- 额外空间：`O(1)`。

也可以使用 `set` 保存访问过的节点，时间 `O(n)`、空间 `O(n)`。

---

11. 链表相交

两条链表可能共享后半部分节点：

```
A1 → A2 ┐
        ├→ C1 → C2
B1 → B2 ┘
```

相交指的是引用同一个节点对象，不是值相等。

一种巧妙方法：两个指针各自到达末尾后，转去另一条链表的头部。

```
[python]
def get_intersection_node(head_a, head_b):
    a = head_a
    b = head_b

    while a is not b:
        a = head_b if a is None else a.next
        b = head_a if b is None else b.next

    return a
```

若两链表长度为 `m n`，两个指针都走过 `m+n` 的对齐路径，最终在交点或共同的 `None` 相遇。

---

12. Python 引用语义中的常见误解

12.1 改变局部变量不等于改变原链表

```
[python]
def skip_head(head):
    head = head.next
```

调用者的变量不会自动改为第二个节点。函数需要返回新的头：

```
[python]
def skip_head(head):
    return head.next
```

12.2 修改节点字段会影响共享结构

```
[python]
head.next = None
```

这是修改节点对象本身，所有引用到该节点的代码都会观察到变化。

12.3 值相等不代表节点相同

```
[python]
node1.val == node2.val
```

不能证明 `node1 is node2`。

---

13. 常见错误

13.1 修改 `.next` 前没有保存后继

可能丢失剩余链表。

13.2 返回旧的 `head`

反转后新头是最后处理的节点，即 `previous`。

13.3 空节点上访问 `.next`

快慢指针条件应为：

快慢指针循环条件应写成 `while fast is not None and fast.next is not None:`。

13.4 删除后错误地前进

删除连续目标节点时，前驱不应立即前进。

13.5 用值判断相交或成环

应比较节点对象身份。

13.6 只画节点，不画引用变量

调试时应同时画出：

```
previous current next_node head tail
```

分别指向哪个节点。

---

14. 本讲检查表

```
1.
2.    next
3.
4.      dummy
5.              next
6.
7.
8.
9.
10.
```

15. 本讲总结

1. 链表节点通过 `.next` 引用连接，不能通过下标随机访问。
2. 修改引用顺序非常重要，必要时先保存后继节点。
3. 虚拟头节点可以统一处理头部插入和删除。
4. 反转链表需要维护已反转部分与未处理部分。
5. 快慢指针可寻找中点并检测环。
6. 链表相交、成环判断关注节点身份，而不是节点值。